DS 2 Informatique - 9 octobre : 4h

Le sujet est décomposé en 4 parties. Vous devez choisir une partie à traiter entre la partie 3 et la partie 4. Votre choix doit être clairement indiqué.

1 Préliminaires

On rappelle quelques notions de graphes :

- Un ensemble I de sommets d'un graphe $G = (S, A)$ est dit indépendant s'il vérifie :

$$\forall (s,t) \in I^2, (s,t) \notin A.$$

- Un ensemble indépendant I est dit maximal pour un graphe G si on ne peut pas ajouter de sommets supplémentaires à I en conservant la notion d'indépendance.
- Un ensemble indépendant I est dit maximum pour un graphe G s'il s'agit d'un plus grand ensemble indépendant de G pour la cardinalité.
- Le graphe complémentaire d'un graphe G est un graphe G' ayant les mêmes sommets et tels que deux sommets distincts de G' soient adjacents si et seulement si il ne le sont pas dans G.
- Un ensemble C de sommets d'un graphe $G = (S, A)$ est une couverture par les sommets s'il vérifie :

$$\forall (s,t) \in A, s \in C \vee t \in C$$

Question 1 : Soit n un entier. Calculer la taille d'un indépendant maximum d'un graphe complet de taille n.

Question 2 : Calculer la taille d'un indépendant maximum d'un cycle de taille n.

Question 3 : Prouver que S' est un ensemble indépendant de $G = (S, A)$ si et seulement si $S \setminus S'$ est une couverture par les sommets.

Dans la suite du sujet, on utilisera le type structuré suivant pour représenter une liste chaînée d'entiers :

```
struct Liste
{
    int val;
    struct Liste* suivant;
};
typedef struct Liste liste;
```

Un graphe G sera représenté par un tableau `G` de listes d'adjacence, c'est-à-dire un objet de type `liste**` de telle sorte que si $s \in S$, alors `G[s]` est la liste chaînée des voisins de s. On associera à ce tableau sa taille `n`.

```
struct Graphe
{
    int n;
    liste** voisins;
};
typedef struct Graphe graphe;
```

On représentera de plus un sous ensemble de S à l'aide d'un tableau de booléen de taille n. La case i du tableau vaudra `true` si et seulement si i appartient au sous-ensemble.

Question 4 : Écrire une fonction `bool* extraire_indep(bool* ensemble, int taille)` qui, étant donné une couverture par les sommets de S, renvoie l'ensemble indépendant associé. Le paramètre `taille` représente le nombre d'éléments de S.

Dans la suite du sujet, on étudiera des méthodes pour construire des ensembles indépendants de taille aussi grande que possible.

2 Algorithmes gloutons

On propose d'étudier un algorithme glouton pour trouver un ensemble indépendant de taille maximume. Celui-ci est décrit dans l'algorithme 1.

Algorithme 1 Glouton(G)

$I \leftarrow \emptyset$
tant que $G \neq \emptyset$ **faire**
  $s \leftarrow$ un sommet de G.
  $I \leftarrow I \cup \{s\}$.
  pour t voisins de s **faire**
    Retirer t de S.
  fin pour
  Retirer s de S.
fin tant que
return I

Question 5 : Démontrer que l'algorithme termine et que l'ensemble renvoyé est bien indépendant.

Question 6 : Démontrer qu'il existe un graphe pour lequel l'algorithme ne renvoie pas une solution optimale.

Dans la suite de la section, on suppose que l'on choisit s de degré minimal. On modifie le type de la section précédente de la façon suivante :

```
struct Graphe
{
    int n;
    liste** voisins;
    bool* tableau;
};

typedef struct Graphe graphe;
```

L'attribut `tableau` de taille n indique si un sommet est toujours présent dans le graphe. Ainsi, si s a été retiré précédemment, `tableau[s]` vaut `false`.

Question 7 : Écrire une fonction `int degre(graphe g, int s)` qui, étant donnés un graphe g et un sommet s, calcule le degré de s dans g. On fera particulièrement attention à ne pas compter les sommets déjà retirés.

Question 8 : Écrire une fonction `int degre_minimal(graphe g)` qui, étant donné un graphe, calcule son sommet de degré minimal.

Question 9 : Écrire une fonction `bool* independant_glouton(graphe g)` qui, étant donné un graphe g, renvoie un ensemble indépendant obtenu par l'algorithme glouton.

Soit k un entier. On considère $G_k = ([\![0, 2k]\!], A_k)$ le graphe défini de la façon suivante :

$$\forall i \in [\![1, k]\!],\ (0, i) \in A_k$$
$$\forall i \in [\![1, k]\!], \forall j \in [\![k+1, 2k]\!]\ (i, j), \in A_k$$
$$\forall i, j \in [\![k+1, 2k]\!],\ (i, j) \in A_k$$

Question 10 : Montrer que G_k admet un ensemble indépendant de taille k alors que l'algorithme glouton renvoie un ensemble de taille 2.

Question 11 : En déduire que le rapport entre la taille de l'ensemble indépendant maximum et celle de l'ensemble indépendant renvoyé est non borné.

3 Graphes bipartis

On rappelle qu'un graphe G est dit biparti si l'existe une partition de ses sommets en deux ensembles telle qu'il n'y ait aucune arête entre deux sommets du même ensemble. Dans cette partie, on restreint notre étude aux graphes G bipartis . On note X et Y la partition des sommets qui en résulte.

3.1 Théorème de König

On cherchera tout d'abord à prouver le résultat suivant, connu sous le nom de théorème de König :

> Pour tout graphe biparti G, le cardinal maximal d'un couplage de G est égal au cardinal minimal d'une couverture par les sommets de G.

Question 12 : Prouver que, dans tout graphe, le cardinal de tout couplage est inférieur ou égal au cardinal de toute couverture par les sommets.

Question 13 : En déduire que, pour tout graphe G, le cardinal maximal d'un couplage est inférieur ou égal au cardinal minimal d'une couverture par les sommets.

Question 14 : Rappeler la définition de chemin anternant associé à un couplage dans un graphe biparti.

Étant donné un couplage C de taille maximale, on note S_0 l'ensemble des sommets de X qui ne sont pas extrémité d'une arête de C. De plus, on note S_1 l'ensemble des sommets de G qui sont soit dans S_0, soit accessibles depuis un sommet de S_0 en suivant un chemin alternant dans G.

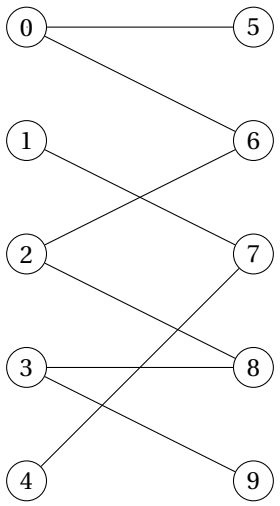

FIGURE 1 – Un exemple de graphe biparti

Question 15 : Proposer un couplage de taille maximale du graphe de la figure 1. Justifier du caractère maximum de celui-ci et caractériser les valeurs de S_0 et S_1 associées.

Question 16 : Montrer que toute arête de C admet soit ses deux extrémités dans S_1, soit aucune.

Question 17 : Montrer que tout sommet de $X \setminus S_1$ est extrémité d'une arête de C.

Question 18 : Montrer que le résultat est aussi vrai pour tout sommet de $Y \cap S_1$.

Question 19 : En déduire que $S_2 = (X \setminus S_1) \cup (Y \cap S_1)$ est une couverture par les sommets de taille $|C|$.

Question 20 : En déduire le théorème de König.

3.2 Implémentation

Dans cette partie, nous allons implémenter des fonctions pour exploiter la démonstration du théorème de König dans un graphe biparti. Plus précisément, nous allons implémenter des algorithmes pour calculer un couplage maximum et une couverture par les sommets, puis déduire un ensemble indépendant maximum.

Nous nous appuierons sur la structure de la section précédente pour représenter les graphes bipartis. L'attribut `tableau` de taille n permet de connaître la bipartition du graphe. Un sommet s appartient à X si et seulement si `part[s]` vaut `true`.

On décide de représenter un couplage avec la même structure de donnée. L'attribut `tableau` permettra d'indiquer si un sommet est extrémité d'une arête du couplage. Si une arête (s,t) est présente dans le couplage, `t` sera le seul élément de `voisins[s]` et réciproquement.

Question 21 : Écrire une fonction `graphe creer_couplage(graphe g)` qui, étant donné un graphe g, crée un couplage vide et le renvoie.

Question 22 : Écrire une fonction `void ajouter_arete(graphe g, int s, int t)` qui, étant donné un graphe g et une arête (s,t) ajoute celle-ci au graphe. On pourra supposer qu'elle n'est pas déjà présente.

Question 23 : Écrire une fonction `void retirer_arete(graphe g, int s, int t)` qui, étant donné un graphe g et une arête (s,t) de celui-ci, la retire. On prêtera attention aux fuites mémoires.

Question 24 : Rappeler comment calculer un couplage maximum dans un graphe biparti.

On représente un chemin à l'aide d'un tableau d'entiers de taille n `succ`. À la case i sera présent le sommet suivant i dans le chemin s'il existe et -1 sinon.

Question 25 : Écrire une fonction `bool parcours_profondeur(graphe g, int s, graphe c, int* chemin, bool derniere_arete)` qui, étant donné un graphe g et un sommet s, réalise un parcours en profondeur en n'exploitant que des chemins alternants dans c. Si le parcours obtient un chemin améliorant, la variable `chemin` sera mise à jour pour en rendre compte. On considérera qu'elle fait bien référence à un espace mémoire alloué de bonne dimension. Le paramètre `derniere_arete` vaudra `true` si et seulement si la dernière arête emprunté appartenait à c.

Question 26 : Écrire une fonction `bool chemin_ameliorant(graphe g, graphe c, int* chemin)` qui, étant donné un graphe g et un couplage c, teste si celui-ci possède un chemin améliorant. Si c'est bien le cas, `chemin` sera modifié en fonction.

Question 27 : Écrire une fonction `void ameliorer(graphe c, int* chemin)` qui, étant donné un couplage c et un chemin supposé améliorant, modifie le couplage pour augmenter son cardinal.

Question 28 : Écrire une fonction `graphe calculer_couplage_max(graphe g)` qui prend en entrée un graphe biparti g et renvoie un couplage maximum.

Question 29 : Écrire une fonction `bool* calculer_couverture_par_sommets(graphe g, graphe c)` qui, étant donné un graphe biparti g et un couplage maximum c, renvoie une couverture minimale par les sommets, représentée par un tableau de booléens.

Question 30 : Implémenter une fonction `bool* calculer_ensemble_independant_maximum(graphe G)` qui prend en entrée un graphe g, et qui renvoie un ensemble indépendant maximum correspondant.

Question 31 : Quel est l'ensemble indépendant obtenu sur le graphe de la figure 1 ?

4 Graphes triangulés

4.1 Théorème de Menger

Soit X et Y deux ensembles de sommets d'un graphe G.

- Un XY-séparateur est un ensemble de sommets tel que le graphe obtenu en supprimant les sommets de cet ensemble ne contient aucun chemin de X à Y. On appelle sa taille le nombre de sommets le composant.
- Un XY-connecteur est un sous-graphe de G tel que chacune de ses composantes est un chemin disjoint des autres de X vers Y. Les chemins doivent même être disjoints en leur extrémités. On appelle taille d'un XY-connecteur le nombre de chemins le composant.

On cherche à prouver le théorème suivant appelé théorème de Menger :

> Pour toute paire d'ensemble de sommets X, Y, la taille minimale d'un XY-séparateur est égale à la taille maximale d'un XY-connecteur.

Moins formellement, le nombre minimal de sommets à retirer à G pour déconnecter X de Y est exactement le nombre de chemins disjoints de X à Y.

Question 12 : Prouver que la taille minimale d'un XY-séparateur est supérieure à la taille maximale d'un XY-connecteur.

On note s la taille minimale d'un XY-séparateur de G et l'on suppose que G contient une arête (x, y). On note G' le graphe obtenu en la retirant de G. On suppose de plus que G' vérifie la propriété que l'on cherche à démontrer.

Question 13 : Montrer que si S' est un XY-séparateur de G' de taille minimale strictement plus petite que s, $R := S' \cup \{x\}$ et $T := S' \cup \{y\}$ sont des XY-séparateurs de G de taille minimale.

Question 14 : Montrer que tout XR-séparateur de G' est un XY-séparateur de G. On admettra qu'il en est de même pour un TY-séparateur de G'.

Question 15 : Prouver qu'il existe un XY-connecteur de G de taille s.

Question 16 : Prouver soigneusement par récurrence le théorème recherché.

4.2 Recherche d'un ensemble indépendant

Un graphe est dit triangulé si tous cycle sans corde possède au plus 3 sommets.
On cherchera dans cette partie, à construire un algorithme polynomial de recherche d'un ensemble indépendant maximum dans de tels graphes.

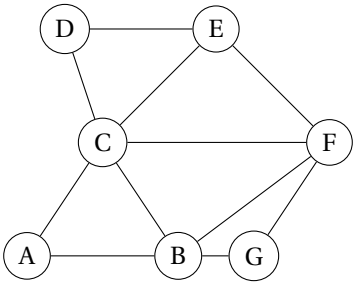

FIGURE 2 – Un graphe triangulé

On dit qu'un sommet s de G est riche si ses voisins sont tous reliés entre eux.

On suppose qu'il existe u et v sont deux sommets non adjacents dans G. On note V_u l'ensemble des voisins de u et V_v l'ensemble des voisins de v. Enfin, on note K un $V_u V_v$-séparateur minimal.

Question 17 : Soit x et y deux sommets de K. Montrer qu'il existe un cycle élémentaire passant par x, y, u et v.

Question 18 : En déduire que tous les sommets de K sont voisins entre eux.

Question 19 : On considère C_u les sommets accessibles depuis u sans passer par K. En étudiant $C_u \cup K$, montrer qu'il existe deux sommets riches non adjacents dans G.

On a ainsi prouvé la propriété suivante :

> Si un graphe triangulé possède deux sommets non adjacents, il existe deux sommets riches non adjacents.

On utilise ici le type de la seconde partie pour représenter des graphes. L'attribut `tableau` indique si un somme a été retiré du graphe.

Question 20 : Écrire une fonction `bool est_riche(graphe g, int s)` qui teste si un sommet s d'un graphe g est riche. On ne prêtera pas attention aux sommet ayant été retirés.

Question 21 : Écrire une fonction `int trouver_riche(graphe g)` qui, étant donné un graphe, en renvoie un sommet riche. On renverra -1 dans le cas où il n'en existe pas.

On propose d'utiliser l'algorithme glouton en sélectionnant à chaque itération un sommet riche.

Algorithme 2 Glouton_triangule(G)

$I \leftarrow \emptyset$
tant que $G \neq \emptyset$ **faire**
  $s \leftarrow$ un sommet riche de G.
  $I \leftarrow I \cup \{s\}$.
  pour t voisins de s **faire**
    Retirer t de S.
  fin pour
  Retirer s de S.
fin tant que
return I

Question 22 : Détailler l'exécution de l'algorithme sur le graphe de la figure 2.

Question 23 : Montrer que la propriété "I peut être complété en un ensemble indépendant maximum" est un invariant de la boucle Tant que.

Question 24 : Prouver que l'algorithme termine et renvoie bien un ensemble indépendant maximum.

Question 25 : Écrire une fonction `bool* independant_triangule (graphe g)` qui étant donné un graphe g triangulé, renvoie un ensemble indépendant maximum.
Question 26 : Analyser la complexité de l'algorithme.